Linear Regression (Simple + Multiple) on `XiaomiMiMo/MiMo-V2.6-RL-oss`

Article ke saare steps is dataset par: **EDA → Simple Linear Regression → Evaluation → Multiple Linear Regression → R^2 / Overfitting check → Cross-validation**.

⚠️ Ye dataset mostly **text** hai (prompts / responses), aur linear regression ko **numbers** chahiye. Isliye notebook text columns se numeric features banata hai (characters, words, lines, digits, code-blocks) aur ek numeric target chunta hai. Aap neeche `TARGET` khud bhi set kar sakte hain.

```
!pip -q install datasets seaborn scikit-learn
```

1. Imports & Data Loading

```
import json
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from datasets import load_dataset
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

SEED = 42
dataset_name = "XiaomiMiMo/MiMo-V2.6-RL-oss"
subset_name = "code"   # 'code', 'cyber', 'general', 'music', 'webdev'

print(f"Fetching '{subset_name}' subset from {dataset_name}... Please wait.")
dataset = load_dataset(dataset_name, subset_name, split="train")
df = pd.DataFrame(dataset)

print("\n✅ Dataset successfully loaded into pandas!")
print(f"Total rows fetched: {len(df)}")
df.head(3)
```

2. Exploratory Data Analysis (original data)

```
print("Shape:", df.shape)
print("\nColumn dtypes:")
print(df.dtypes)
print("\nMissing values per column:")
print(df.isna().sum())
df.head()
```

3. Text → Numeric features

Text/list/dict columns se numeric features banate hain, taaki regression ho sake.

```
def to_text(x):
    if x is None:
        return ""
    if isinstance(x, str):
        return x
    try:
        return json.dumps(x, ensure_ascii=False, default=str)
    except Exception:
        return str(x)

feat = pd.DataFrame(index=df.index)
text_cols = []
for c in df.columns:
    if pd.api.types.is_bool_dtype(df[c]):
        feat[c] = df[c].astype(int)
    elif pd.api.types.is_numeric_dtype(df[c]):
        feat[c] = df[c]
    else:
        s = df[c].map(to_text)
        text_cols.append(c)
        feat[f"{c}__chars"]  = s.str.len()
        feat[f"{c}__words"]  = s.str.split().str.len()
        feat[f"{c}__lines"]  = s.str.count("\n") + 1
        feat[f"{c}__digits"] = s.str.count(r"\d")
```

```
            feat[f"{c}__code_fences"] = s.str.count("```")

    feat = feat.replace([np.inf, -np.inf], np.nan).astype(float)
    # constant columns kisi kaam ke nahi
    feat = feat.loc[:, feat.nunique(dropna=True) > 1]
    print("Text columns:", text_cols)
    print("Numeric features created:", list(feat.columns))
    feat.head()
```

4. Target chunna

Agar aap khud target dena chahte hain to `TARGET = "column_name"` likh dein (jaise `"reward"` ya `"response__chars"`). Warna auto-select hoga.

```
    TARGET = None   # e.g. "response__chars"

    if TARGET is None:
        prefs = ["reward", "score", "label", "response", "answer", "output",
                 "completion", "solution", "prompt", "question", "input"]
        for p in prefs:
            hits = [c for c in feat.columns if p in c.lower() and (c.endswith("__chars") or "__" not in c)]
            if hits:
                TARGET = hits[0]
                break
        if TARGET is None:
            TARGET = feat.columns[-1]

    source = TARGET.split("__")[0]
    # leakage se bachne ke liye same source column ke features hata do
    FEATURES = [c for c in feat.columns if c != TARGET and not c.startswith(source + "__") and c != source]
    print("TARGET  :", TARGET)
    print("FEATURES:", FEATURES)
    assert len(FEATURES) >= 1, "Features nahi bache - TARGET change karein."

    data = feat[FEATURES + [TARGET]].dropna().copy()
    print("Rows used:", len(data))

    # Plots tez rakhne ke liye sample
    plot_df = data.sample(min(len(data), 3000), random_state=SEED)
```

5. EDA on numeric data

```
    print("Shape:", data.shape)
    print(data.describe().round(2).T)
```

```
    correlations = data.corr()
    plt.figure(figsize=(max(6, len(data.columns)*0.9), max(5, len(data.columns)*0.7)))
    sns.heatmap(correlations, annot=True, fmt=".2f", cmap="coolwarm").set(title="Heat map - Pearson Correlations")
    plt.show()

    print("Correlation with target:")
    print(correlations[TARGET].drop(TARGET).sort_values(key=abs, ascending=False))
```

```
    for var in FEATURES:
        plt.figure(figsize=(6, 4))
        sns.regplot(x=var, y=TARGET, data=plot_df, scatter_kws={"alpha": 0.3, "s": 10}, line_kws={"color": "red"}) \
           .set(title=f"Regression plot of {var} and {TARGET}")
        plt.show()
```

Outliers check (boxplots) — linear models outliers se sensitive hote hain

```
    n = len(data.columns)
    cols = 3
    rows = int(np.ceil(n / cols))
    fig, axes = plt.subplots(rows, cols, figsize=(5*cols, 3.5*rows))
    for ax, c in zip(np.array(axes).ravel(), data.columns):
        sns.boxplot(x=data[c], ax=ax); ax.set_title(c)
    for ax in np.array(axes).ravel()[n:]:
        ax.axis("off")
    plt.tight_layout(); plt.show()
```

6. Simple Linear Regression (1 feature)

Sabse zyada correlated feature use hoga.

```
best_feature = correlations[TARGET].drop(TARGET).abs().idxmax()
print("Best single feature:", best_feature, "| corr =", round(correlations.loc[best_feature, TARGET], 4))

y = data[TARGET].values.reshape(-1, 1)
X = data[best_feature].values.reshape(-1, 1)
print(X.shape, y.shape)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)

regressor = LinearRegression()
regressor.fit(X_train, y_train)
print("Intercept:", regressor.intercept_)
print("Slope    :", regressor.coef_)
```

```
y_pred = regressor.predict(X_test)
df_preds = pd.DataFrame({"Actual": y_test.squeeze(), "Predicted": y_pred.squeeze()})
print(df_preds.head(10))

plt.figure(figsize=(7, 5))
plt.scatter(X_test, y_test, alpha=0.3, s=10, label="Actual")
xs = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
plt.plot(xs, regressor.predict(xs), color="red", label="Fitted line")
plt.xlabel(best_feature); plt.ylabel(TARGET); plt.legend(); plt.title("Simple Linear Regression")
plt.show()
```

Evaluation (MAE, MSE, RMSE, R^2)

```
def evaluate(y_true, y_hat, name=""):
    mae = mean_absolute_error(y_true, y_hat)
    mse = mean_squared_error(y_true, y_hat)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_true, y_hat)
    print(f"--- {name} ---")
    print(f"Mean absolute error     : {mae:.2f}")
    print(f"Mean squared error      : {mse:.2f}")
    print(f"Root mean squared error : {rmse:.2f}")
    print(f"R2                      : {r2:.4f}")
    return mae, mse, rmse, r2

res_simple = evaluate(y_test, y_pred, "Simple LR (test)")
print("Train R2:", round(regressor.score(X_train, y_train), 4), "| Test R2:", round(regressor.score(X_test, y_test), 4))
print("Target mean for reference:", round(float(data[TARGET].mean()), 2), "| std:", round(float(data[TARGET].std()), 2))
```

7. Multiple Linear Regression (saare features)

```
y = data[TARGET]
X = data[FEATURES]
print(X.shape)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)

regressor = LinearRegression()
regressor.fit(X_train, y_train)
print("Intercept:", regressor.intercept_)

coefficients_df = pd.DataFrame(data=regressor.coef_, index=X.columns, columns=["Coefficient value"])
print(coefficients_df)
```

```
y_pred = regressor.predict(X_test)
results = pd.DataFrame({"Actual": y_test, "Predicted": y_pred})
print(results.head(10))

res_multi = evaluate(y_test, y_pred, "Multiple LR (test)")
```

R^2 manually + Train vs Test (overfitting check)

```
actual_minus_predicted = sum((y_test - y_pred) ** 2)
actual_minus_actual_mean = sum((y_test - y_test.mean()) ** 2)
r2_manual = 1 - actual_minus_predicted / actual_minus_actual_mean
print("R2 (manual)      :", r2_manual)
```

```
print("R2 (sklearn)     :", regressor.score(X_test, y_test))

train_r2 = regressor.score(X_train, y_train)
test_r2 = regressor.score(X_test, y_test)
print(f"Train R2: {train_r2:.4f} | Test R2: {test_r2:.4f}")
if train_r2 - test_r2 > 0.1:
    print("⚠️ Train R2 test R2 se kaafi zyada hai -> overfitting ka shak.")
else:
    print("✅ Train aur Test R2 qareeb hain -> overfitting ka khaas sign nahi.")
```

8. Cross-validation (article ka suggestion)

```
cv_r2 = cross_val_score(LinearRegression(), X, y, cv=5, scoring="r2")
cv_rmse = -cross_val_score(LinearRegression(), X, y, cv=5, scoring="neg_root_mean_squared_error")
print("5-fold R2   :", cv_r2.round(4), "| mean =", cv_r2.mean().round(4))
print("5-fold RMSE :", cv_rmse.round(2), "| mean =", cv_rmse.mean().round(2))
```

9. Outliers handle karke dobara (IQR clipping)

```
def clip_iqr(frame):
    out = frame.copy()
    for c in out.columns:
        q1, q3 = out[c].quantile([0.25, 0.75])
        iqr = q3 - q1
        out[c] = out[c].clip(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
    return out

data_c = clip_iqr(data)
Xc, yc = data_c[FEATURES], data_c[TARGET]
Xc_train, Xc_test, yc_train, yc_test = train_test_split(Xc, yc, test_size=0.2, random_state=SEED)
reg_c = LinearRegression().fit(Xc_train, yc_train)
res_clip = evaluate(yc_test, reg_c.predict(Xc_test), "Multiple LR after outlier clipping (test)")
print("Train R2:", round(reg_c.score(Xc_train, yc_train), 4), "| Test R2:", round(reg_c.score(Xc_test, yc_test), 4))
```

10. Summary

```
summary = pd.DataFrame(
    [res_simple, res_multi, res_clip],
    columns=["MAE", "MSE", "RMSE", "R2"],
    index=["Simple LR", "Multiple LR", "Multiple LR (outliers clipped)"]
).round(4)
summary
```

Conclusion

- Simple LR ek feature se, Multiple LR saare features se fit hua; dono ko MAE/MSE/RMSE/R^2 se naapa gaya.
- Agar R^2 kam hai to iska matlab hai text-length jaise features target ko achhe se explain nahi karte (ya relationship non-linear hai) — ye dataset asal mein regression ke liye bana hi nahi.
- Behtar nateeje ke liye: koi asli numeric column (jaise reward/score, agar maujood ho) `TARGET` mein dein, ya zyada features banayein.